

Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 11

Funzioni ricorsive

Dal meccanismo delle chiamate di funzione si deduce che :

- All'interno di una funzione posso richiamarne un'altra.
- Nessun problema a richiamare all'interno di una funzione un'altra istanza della stessa! → **funzioni ricorsive**
- **Simulano un processo iterativo**

Iterazione e ricorsione

Una funzione *ricorsiva* chiama se stessa direttamente o indirettamente

ESEMPIO: calcolo del fattoriale $n!$

$0!$ e $1!$ sono per definizione uguali a 1.

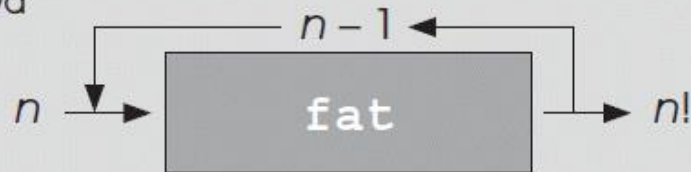
$n! = n \cdot (n - 1) \cdot (n - 2) \cdot (n - 3) \cdot \dots \cdot 2 \cdot 1$ (def. iterativa)

$n! = n \cdot (n - 1)!$ (def. ricorsiva)

versione iterativa



versione ricorsiva



Iterazione e ricorsione

Funzione *iterativa*

(usa un ciclo)

```
int fatIte(int n)
{
    int fat;
    if(n==0)
        fat = 1;
    else
        for(fat=n; n>2; n--)
            fat = fat*(n-1);
    return fat;
}
```

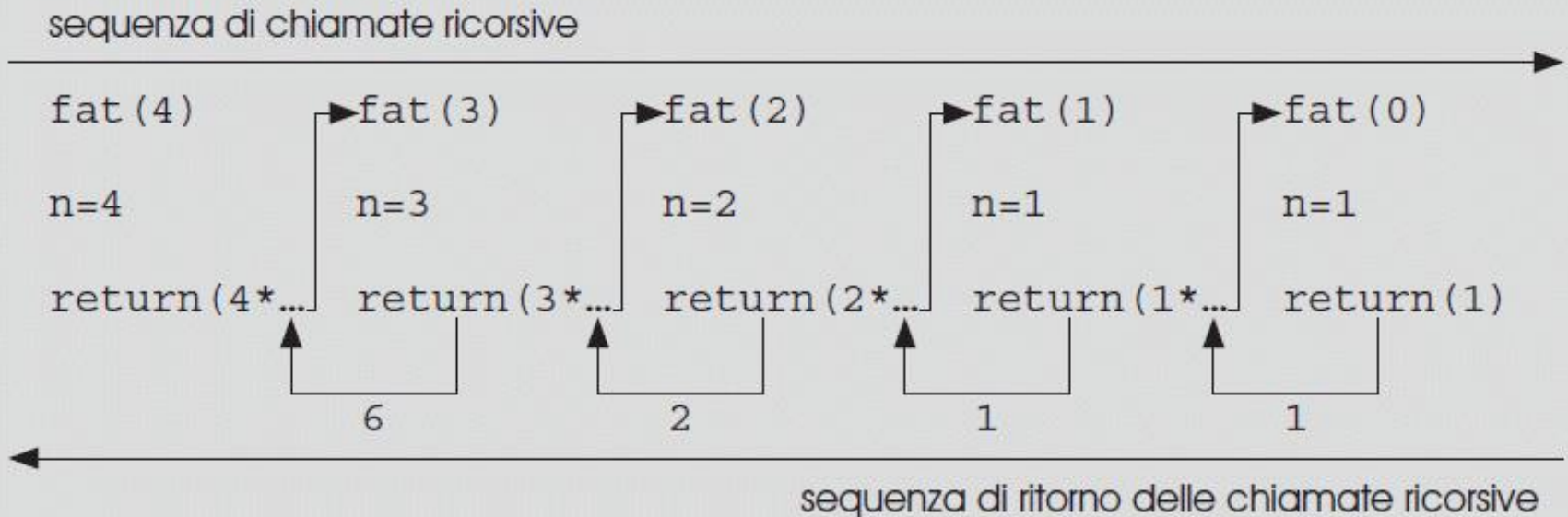
Funzione *ricorsiva*

(chiama se stessa)

```
int fatRic(int n)
{
    if(n==0)
        return(1);
    else
        return(n*fatRic(n-1));
}
```

Iterazione e ricorsione

```
int fat(int n) {  
    if(n==0)  
        return(1);  
    else  
        return(n*fat(n-1));  
}
```



Ricorsione

Quando una funzione ricorsiva invoca se stessa, si eseguono le stesse azioni che vengono eseguite quando viene invocata una funzione qualsiasi:

1. Si sospende l' esecuzione della funzione invocante
2. Si esegue la funzione invocata fino alla sua terminazione
3. Si riprende l' esecuzione della funzione invocante dal punto in cui era stata sospesa

Esempio: VerificaRicorsione.c

```
#include <stdio.h>
#include <stdlib.h>

void verifica(){
    int input;
    printf( "? " );
    scanf("%d", &input);
    if ( input != 0 ) {
        verifica();
        printf( "%d\n", input );
    };
};

int main(){
    verifica();
    return(0);
}
```

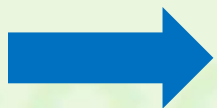
Scrivere una funzione ricorsiva

Esistono due regole ben definite che vanno utilizzate per scrivere funzioni ricorsive:

1. individuare il caso base
soluzione del problema in almeno un caso particolare, senza ricorrere ad una chiamata ricorsiva (possono esserci più casi base)
2. definire il passo ricorsivo
deve effettuare la chiamata ricorsiva semplificando il problema (ossia avvicinandosi al caso base)

Se non scriviamo bene il programma, le chiamate ricorsive si possono eseguire una dopo l'altra, all'infinito. **Invece, se :**

1. ad ogni invocazione il problema diventa sempre più semplice e si avvicina al caso base
2. la soluzione del caso base non richiede ricorsione allora, per quanto complesso possa essere il problema, certamente la soluzione viene calcolata in un numero finito di passi.



La soluzione ricorsiva di un problema è un algoritmo effettivo in quanto arriva a conclusione in un numero finito di passi

Scrivere una funzione ricorsiva

Esistono due regole ben definite che vanno utilizzate per scrivere funzioni ricorsive:

1. individuare il caso base
soluzione del problema in almeno un caso particolare, senza ricorrere ad una chiamata ricorsiva (possono esserci più casi base)
2. definire il passo ricorsivo
deve effettuare la chiamata ricorsiva semplificando il problema (ossia avvicinandosi al caso base)

Quando utilizzare la ricorsione

PRO

- Spesso la ricorsione permette di risolvere un problema anche molto complesso con poche linee di codice

CONTRO

- La ricorsione è poco efficiente perché richiama molte volte una funzione
- richiede tempo per la gestione dello stack (allocare e passare i parametri, salvare l'indirizzo di ritorno, e i valori di alcuni registri della CPU)
- consuma molta memoria (alloca un nuovo stack frame ad ogni chiamata, definendo una nuova ulteriore istanza delle variabili locali e dei parametri ogni volta)

Esempio da evitare:

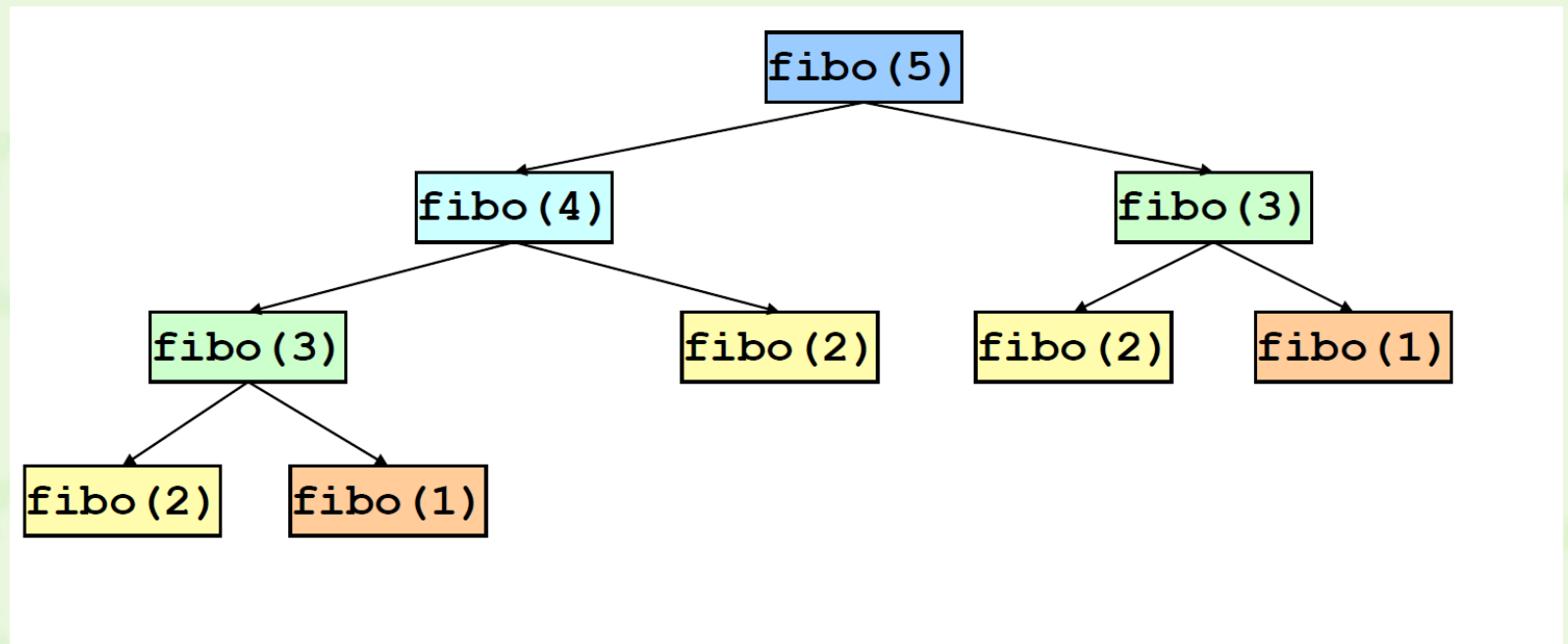
```
long fibo(long n)
{
    if (n<=2)
        return 1;
    else
        return fibo(n-1) + fibo(n-2);
}
```



Ogni chiamata genera altre 2 chiamate,
In totale vengono effettuate 2^n chiamate

complessità esponenziale!!!

`fibo(n)` chiama `fibo(n-1)` e `fibo(n-2)`,
anche `fibo(n-1)` chiama `fibo(n-2)`, ecc.:
si hanno calcoli ripetuti, **inefficiente!**



Ogni algoritmo ricorsivo si può
riscrivere in un equivalente
algoritmo iterativo.

Un esempio che non può mancare: La torre di Hanoi



https://it.wikipedia.org/wiki/Torre_di_Hanoi

<https://www.geogebra.org/m/fN74ZAsj>

REGOLE

Si hanno a disposizione 3 pile di dischi (*torri*) allineate:

- all'inizio tutti i dischi si trovano sulla pila di sx



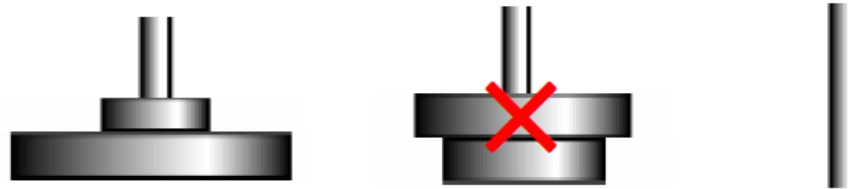
- alla fine tutti i dischi si devono trovare sulla pila di dx

I dischi sono tutti di dimensione diversa e quando si trovano su una pila devono rispettare la seguente regola

nessun disco può avere sopra di sé dischi più grandi

REGOLE

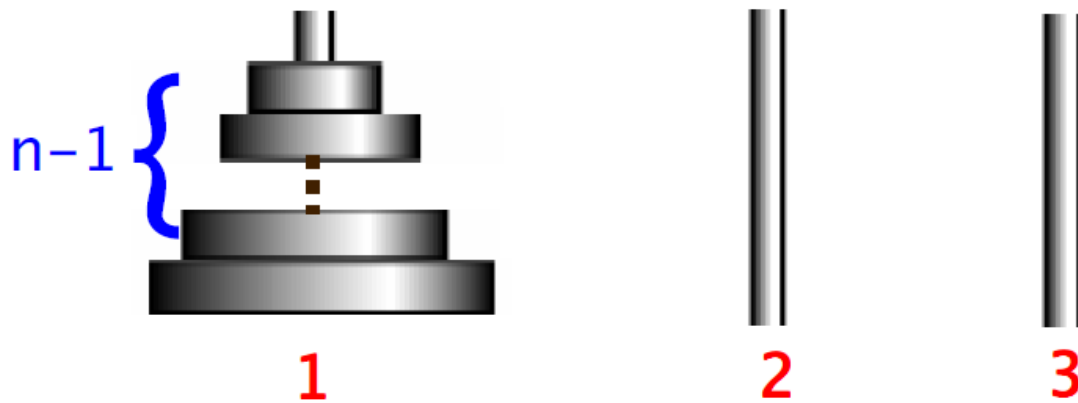
1. si può spostare un disco alla volta
2. il disco da spostare deve essere rimosso dalla cima della torre ed inserito in cima ad un'altra torre
non può essere parcheggiato all'esterno
3. in ogni momento deve essere sempre soddisfatta la condizione che *nessun disco può avere sopra di sé dischi più grandi*

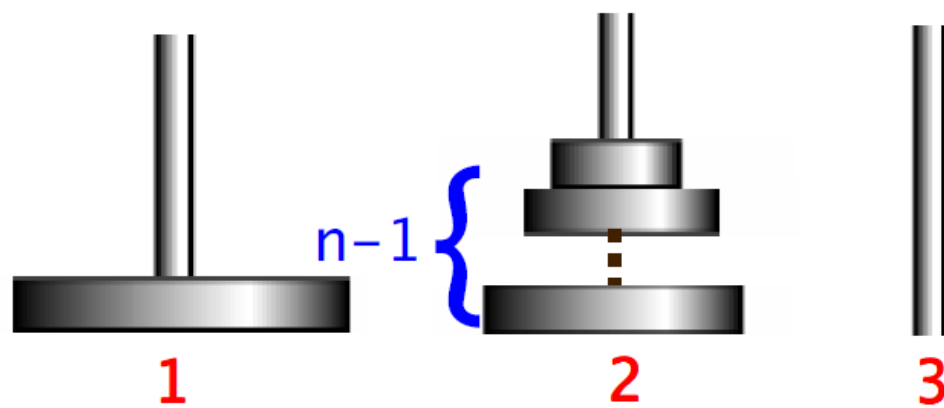


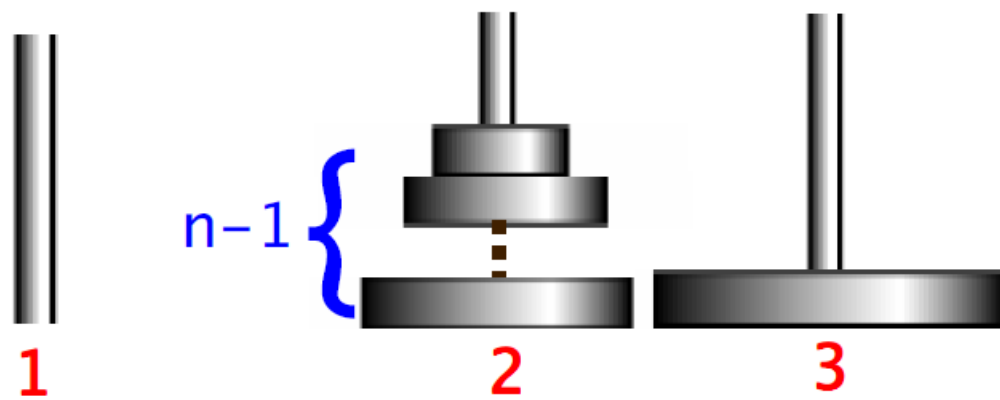
Soluzione

- Per la soluzione del gioco di Hanoi è noto un algoritmo ricorsivo
- Il problema generale consiste nello spostare n dischi da una torre all'altra, usando la terza torre come deposito temporaneo
- Per spostare n dischi da una torre all'altra si suppone di saper spostare $n - 1$ dischi da una torre all'altra
- Per descrivere l'algoritmo identifichiamo le torri con i numeri interi 1,2,3

Per spostare n dischi dalla torre 1 alla torre 3:

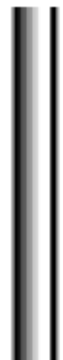




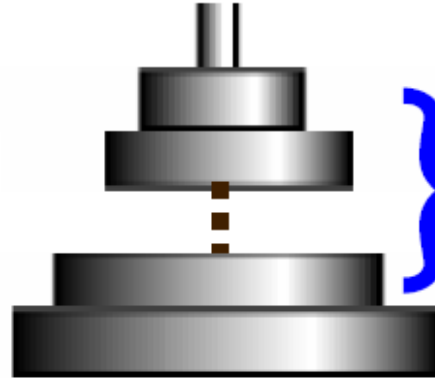




1



2



3

Per spostare n dischi dalla torre 1 alla torre 3:

1. gli $n - 1$ dischi in cima alla torre 1 vengono spostati sulla torre 2, usando la torre 3 come deposito temporaneo (si usa una chiamata ricorsiva, al termine della quale la torre 3 rimane vuota)
2. il disco rimasto sulla torre 1 viene spostato nella torre 3
3. gli $n - 1$ dischi in cima alla torre 2 vengono spostati sulla torre 3, usando la torre 1 come deposito temporaneo (si usa una chiamata ricorsiva, al termine della quale la torre 1 rimane vuota)

Il caso base si ha quando $n = 1$ e l'algoritmo usa il solo passo 2, che non ha chiamate ricorsive

Si dimostra che il numero di mosse per risolvere il gioco con questo algoritmo è $2^n - 1$

Complessità dell'algoritmo:

Proposizione: L'algoritmo ricorsivo prima descritto esegue $2^n - 1$ mosse per spostare n dischi dalla torre 1 alla torre 3.

Dimostrazione: (per induzione)

- $n=1$: $2^1 - 1 = 1$ ok
- Assumo che occorrono $2^{n-1} - 1$ mosse per spostare $n-1$ dischi.
- Per spostare n dischi:

$$2^{n-1} - 1 + 1 + 2^{n-1} - 1 = 2^n - 1$$

Qualche conto...

Numero dischi	Numero mosse	tempo
4	15	15 sec
8	255	4 min e 15 sec
12	4.095	1 ora 8 min e 15 sec
16	65.535	18 ore 12 min e 15 sec
24	16.777.216	194 gg 4 ore 20 min 15 sec
64	18.446.744.073.709.551.615	5.845.580.504 secoli circa

Soluzione in C

```
void muoviUnDisco(int sorgente, int destin)
{
    printf(" muovi un disco da %2d a %2d\n", sorgente, destin);
} /* muoviUnDisco */
```

```
void muovi(int n, int sorgente, int destin, int aus)
{
    if (n == 1)
        muoviUnDisco(sorgente, destin);
    else {
        muovi(n-1, sorgente, aus, destin);
        muoviUnDisco(sorgente, destin);
        muovi(n - 1, aus, destin, sorgente);
    }
} /* muovi */
```

Nel main:

```
muovi(dischi, 1, 3, 2);
```

Vediamo l'algoritmo scritto in C
`hanoi.c`